

# Inference System Design and Performance Optimization

pig7selene

September 5, 2026

## Abstract

An LLM serving system is optimized as an end-to-end pipeline, not by accumulating isolated execution tricks. This chapter turns the mechanisms of Chapters 19–24 into a design methodology: characterize the request distribution and service-level objectives, identify the active bottleneck by phase, select one compatible intervention, and validate both performance and model behavior. It relates Roofline-style arithmetic intensity, cache capacity, batching, quantization, speculative decoding, and distributed placement to capacity planning, tail latency, cost per token, and repeatable regression testing.

## 1 Introduction

The previous six chapters established the components of modern LLM serving. A request is Prefilled, assigned persistent KV Cache state, scheduled through incremental Decode, and eventually completed and reclaimed. Its weights can be quantized, its serial Decode loop can be accelerated by speculation, and its computation can span a distributed execution group. None of these mechanisms is an optimization strategy by itself. A modification that improves one measurement can worsen a different measurement that the application actually values.

For example, a larger batch can raise aggregate token throughput while increasing queueing delay and Time to First Token (TTFT). Quantization can reduce weight traffic but add dequantization work, require a different kernel, or exceed a quality tolerance. Speculative Decoding can reduce expensive target-model iterations while paying for drafting, verification, and provisional cache state. More accelerators can increase capacity or make a model fit while introducing communication on every Decode step. The useful question is therefore not “which technique is fastest?” but “which constraint is active for this workload and service objective?”

This closing chapter of the Inference and Serving section develops a measurement-driven answer. It does not rederive attention caching, continuous batching, quantization formats, speculative acceptance, or distributed collectives. Chapters 19–24 remain the sources for those mechanisms. Here they become controlled choices in an end-to-end system design.

## 2 The End-to-End Serving Path

An inference service has an execution path before any Transformer kernel runs. A request arrives with a prompt, generation policy, and often a service class. It can wait in a queue, be admitted only if resources are available, run Prefill, acquire or extend KV Cache blocks, join Decode scheduling iterations, stream tokens, and finally release its state. The following diagram gives the logical path; an implementation may combine or distribute its stages, but each boundary has an observable cost and ownership rule.

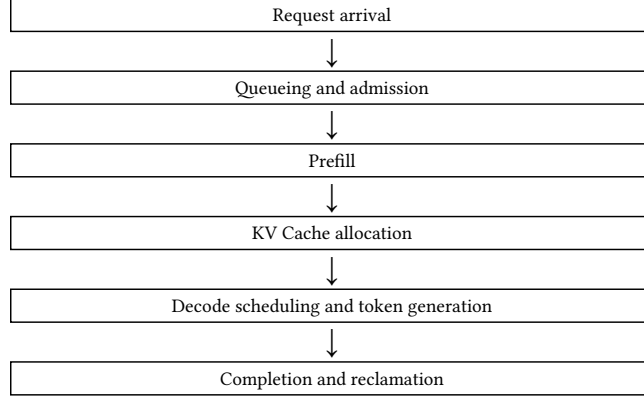


Figure 1: The end-to-end serving path combines request management with model execution. A performance result is interpretable only when it states which of these boundaries are included in its measurements.

Figure 1 separates two kinds of work. Queueing, admission, cache allocation, and reclamation are control-plane or resource-management decisions. Prefill and Decode are model execution phases. Yet the distinction is not a boundary between important and unimportant work: an efficient kernel cannot repair a high TTFT caused by an overloaded queue, and an idle accelerator cannot serve a request whose cache reservation would exceed memory. Chapter 22 develops these scheduler and allocation interactions; the design task is to observe them together.

For request  $i$ , let  $a_i$  be its arrival time,  $t_i^{\text{first}}$  the time of its first delivered token, and  $t_i^{\text{last}}$  the time of its final delivered token. The quantities

$$\text{TTFT}_i = t_i^{\text{first}} - a_i, \quad L_i^{\text{end}} = t_i^{\text{last}} - a_i \quad (1)$$

have different diagnostic value. TTFT contains queueing and prompt processing under the declared measurement boundary. End-to-end latency additionally includes the generated continuation and its repeated Decode steps. TPOT, defined in Chapter 22, describes the intervals after the first token. Reporting only one of these quantities hides where the request spent time.

### 3 Workload and Service Objectives

Optimization begins with the workload rather than an average request. Let  $S$  be prompt length,  $G$  generated-token count, and  $A$  request arrival rate over a measurement window. A workload is characterized by distributions of  $(S, G, A)$ , not only their means. It also includes concurrency, context limits, sampling controls, model revision, precision, request priorities, cancellation behavior, and the proportion of long-running streams. These variables determine both execution and scheduling: a long prompt changes Prefill work, a long output extends cache lifetime, and a burst of arrivals can make queueing dominate otherwise fast kernels.

The mean can be actively misleading. A small population of requests with very long prompts or continuations may dominate KV Cache occupancy, reserve the scheduler’s active slots, and create the p99 latency seen by users. A short offline benchmark can therefore suggest that a configuration is lightly loaded while a production-like length distribution makes it memory-capacity-bound. When requests share fixed system prompts, prefix reuse can change the distribution of allocated cache bytes without changing the distribution of raw prompt tokens; Chapter 20 explains why these are separate facts.

Service-level objectives (SLOs) make such observations actionable. An interactive assistant may constrain p95 TTFT and p95 TPOT. A background workload may tolerate high latency but require high aggregate output throughput. A constrained deployment may prioritize maximum admitted context, cost per output token, or predictable tail behavior. These are not interchangeable objectives. A latency objective specifies a delay target; a throughput objective specifies delivered work per time;

a capacity objective specifies the state and concurrency that can be admitted; and a cost objective specifies resources consumed per useful output.

For a random metric  $Z$ , its  $p$ th percentile is a value  $z_p$  such that approximately a fraction  $p$  of observations are at most  $z_p$ . Thus p50 describes a typical request, while p95 and p99 expose the tail. Percentiles should be paired with the request population, time window, concurrency, and treatment of rejected, cancelled, or timed-out requests. Omitting difficult requests can make a tail metric look excellent while describing a different service.

| Objective            | Representative metric                                 | Question it answers                                                                 |
|----------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------|
| Interactive latency  | p50, p95, or p99 TTFT and TPOT                        | How quickly does a user receive the first and subsequent tokens?                    |
| Aggregate throughput | Input/output tokens per second; requests per second   | How much declared work does the deployment complete under the workload?             |
| Admission capacity   | Active requests, context budget, KV Cache utilization | How much state can remain live without allocation failure or unsafe overcommitment? |
| Cost efficiency      | Accelerator time or cost per delivered output token   | How efficiently are resources converted into useful output under the SLO?           |

Table 1: A serving configuration must be evaluated against an explicitly chosen objective. No row is implied by success in another row.

## 4 Bottleneck Classification

A bottleneck is the resource or dependency that limits the declared objective in the measured regime. It is not a permanent property of a model. Prefill and Decode can occupy different regimes for the same request; a small batch and a large batch can occupy different regimes on the same accelerator. The classification below guides investigation rather than replaces profiling.

**Compute-bound** execution is limited mainly by available arithmetic throughput. Increasing useful matrix-work occupancy, reducing avoidable operations, or changing the arithmetic kernel can help if data movement and scheduling do not become the next limit. Prefill often has relatively high arithmetic intensity because many known prompt positions are processed together, but this tendency depends on prompt lengths, batch construction, model shape, attention implementation, and hardware.

**Memory-bandwidth-bound** execution is limited by moving weights, activations, or KV state. Decode frequently falls into this regime because it performs modest new-token arithmetic while repeatedly reading large persistent state. Weight quantization can reduce transferred bytes; cache layout or head sharing can change KV traffic; batching can alter reuse and kernel efficiency. None guarantees a speedup if unpacking, dequantization, or another resource becomes limiting.

**Memory-capacity-bound** execution cannot admit the desired model state, context, or concurrency. Capacity is distinct from bandwidth: an accelerator can have enough bandwidth to process an already-admitted request while lacking enough memory to keep more cache blocks live. Paging, prefix sharing, cache quantization, shorter limits, additional replicas, or a different model layout are possible responses, each with semantic or systems costs. Chapter 20 gives the cache accounting that should precede these choices.

**Communication-bound** execution spends its critical path waiting for collective operations, inter-stage transfers, or cache handoffs. This regime arises in distributed layouts when Tensor, Pipeline, or Expert Parallel groups cross an unfavorable link or when a small Decode workload cannot hide communication. Adding devices can reduce local memory yet increase token latency. Chapter 24 explains these ownership and topology trade-offs.

**Scheduling-bound** execution has provisioned arithmetic and memory resources that are poorly converted into useful work because of queueing, fixed batch membership, fragmentation, admission policy, or phase interference. Continuous batching and cache-aware scheduling address this class, but only if the measured delay is actually waiting or underoccupancy rather than a slow model kernel. ORCA’s iteration-level scheduling illustrates why autoregressive request lifetimes require a different batching granularity from one-shot inference [1].

## 5 Arithmetic Intensity and Roofline Intuition

A concise way to relate computation and data movement is **arithmetic intensity**. Let  $F$  be the number of relevant arithmetic operations and let  $Q$  be the number of bytes moved across the limiting memory boundary. Then

$$I = \frac{F}{Q} \quad (2)$$

is the arithmetic intensity in operations per byte. If  $\Pi_{\text{peak}}$  is a device’s attainable arithmetic ceiling and BW is its attainable bandwidth for the relevant data path, the Roofline-style bound is

$$\Pi \leq \min(\Pi_{\text{peak}}, I \text{ BW}). \quad (3)$$

Williams, Waterman, and Patterson introduced this model as a way to expose whether a computation is constrained by arithmetic capability or data movement [2]. Here Equation 3 is an intuition, not a promise of measured performance. Real LLM serving includes multiple memory levels, irregular cache accesses, kernel launch and synchronization costs, communication, queueing, and finite batch effects. Still, it prevents a common category error: a reduction in FLOPs matters most when arithmetic is limiting, whereas a reduction in bytes matters most when bandwidth is limiting.

The contrast between phases follows naturally. Prefill can increase  $F$  substantially while also reusing weights and activating efficient matrix operations over many prompt tokens. Decode adds little new-token work per layer but reads weights and an ever-growing history of keys and values. Quantization primarily reduces stored and transferred bytes; batching can change both the effective reuse and the queueing cost; KV Cache management changes persistent state capacity and access patterns. Speculative Decoding changes the number and shape of target-model evaluations rather than simply reducing a fixed count of FLOPs. Distributed execution introduces an additional communication boundary that Equation 3 does not represent. Pope et al. analyze these phase- and partition-dependent inference trade-offs for very large Transformers [3].

## 6 Selecting Compatible Optimization Levers

The mechanisms of Chapters 19–24 should be chosen from an observed bottleneck, not stacked by default. Table 2 summarizes their primary pressure and the condition that can reverse their apparent benefit.

| Lever                                    | Primary pressure addressed                                              | Condition that must be measured                                                                    |
|------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| KV Cache paging, sharing, or compression | Capacity waste, fragmentation, or long-context cache footprint          | Cache traffic, allocation metadata, effective context, and quality remain acceptable.              |
| Weight quantization                      | Model memory and weight bandwidth                                       | Kernel path, dequantization overhead, and output-quality tolerance justify the lower precision.    |
| Continuous batching                      | Idle execution capacity from variable request lengths                   | Extra queueing or phase interference does not violate TTFT or TPOT targets.                        |
| Speculative Decoding                     | Serial target-model Decode iterations                                   | Acceptance, draft cost, verification efficiency, and provisional state produce an end-to-end gain. |
| Distributed inference                    | Single-device model capacity or insufficient aggregate service capacity | Communication, topology, load balance, and cache placement do not dominate the critical path.      |

Table 2: Each serving technique changes a different resource relationship. The primary pressure identifies where to test first, not a guarantee that the technique improves every workload. KV Cache optimization becomes central when long contexts or concurrent requests exhaust allocator capacity, not merely when the model’s weight file is large. Paged allocation and prefix sharing can reduce reservation and duplication waste, while eviction or lossy cache compression changes what information remains available to attention. Weight quantization is most compelling when weights consume scarce memory or repeated weight reads constrain Decode. Chapter 21 explains why a smaller representation must still be evaluated through the actual kernel and quality boundary.

Continuous batching increases useful occupancy across heterogeneous generation lengths, but it cannot change the cost of a particular Decode step. Its central trade-off is service policy: more work per batch can improve aggregate throughput while delaying newly arrived requests or disrupting active streams. Speculative Decoding is especially relevant when target Decode is a serial, expensive bottleneck and a cheap proposal path is accepted often enough; Chapter 23 emphasizes that acceptance rate alone is insufficient. Distributed inference is justified when a model cannot fit, one group cannot supply the required capacity, or a phase-specific placement is beneficial. Its added devices are not free throughput: collectives, stage transfers, and cache handoffs must appear in the phase-specific latency measurement.

## 7 Profiling, Capacity Planning, and Cost

Profiling locates the active constraint before an optimization is selected. At minimum, a trace should separate queueing delay, admission delay, Prefill execution, Decode execution, kernel time, communication time, batch occupancy, and KV Cache occupancy. Accelerator utilization is useful but incomplete: high utilization can reflect productive work, cache traffic, communication waits, or a batch policy that sacrifices interactive latency. A profile should identify which phase is slow, which resource is saturated, and which requests form the tail.

Capacity planning then combines workload measurements with empirical benchmarks. If requests arrive at rate  $\lambda$  and generate a random  $G$  output tokens, the mean offered output-token rate is  $\lambda E[G]$ . If  $C_{\text{out}}$  is the measured sustainable output-token rate of one execution group under the same prompt distribution, context limits, sampling policy, and SLO, a necessary average-load condition is

$$\lambda E[G] < n_R C_{\text{out}}, \quad (4)$$

where  $n_R$  is the number of independent execution-group replicas. Equation 4 is not a sizing formula. It omits tail lengths, Prefill demand, burstiness, queueing discipline, cache growth, and failover headroom. Its purpose is to show why request rate alone is inadequate: a service whose requests generate twice as many tokens needs roughly twice the output work even at the same arrival rate. A credible plan estimates both prompt and output token distributions, model and workspace memory, cache demand, measured phase throughput, and the desired headroom, then validates the resulting deployment under a representative arrival trace.

Cost efficiency uses the same denominator discipline. If an experiment consumes accelerator-time cost  $C_{\text{acc}}$  while delivering  $N_{\text{out}}$  output tokens, then

$$c_{\text{token}} = \frac{C_{\text{acc}}}{N_{\text{out}}} \quad (5)$$

is a conceptual cost per output token.  $C_{\text{acc}}$  may represent priced accelerator time or an internal resource accounting; it need not assume a particular cloud price. Higher useful utilization can lower Equation 5, but maximizing utilization alone may delay interactive requests past their SLO. Costs should therefore be reported alongside the latency distribution and quality checks that define useful service, not as an isolated throughput contest.

## 8 Regression Testing and the Optimization Workflow

An optimization is only a change relative to a controlled baseline. A regression suite should retain the model checkpoint, tokenizer and prompt construction, precision, hardware topology, cache policy, prompt-length distribution, output-length limits, concurrency, sampling parameters, and scheduling configuration. It should measure p50/p95/p99 TTFT and TPOT, end-to-end latency, throughput, queueing, cache utilization, and relevant resource counters. It should also check output behavior: deterministic requests can compare tokens or logits within a declared numerical tolerance, while stochastic requests need compatible seeds, distributions, or task-level quality evaluation.

The optimization loop is deliberately narrow. First define the workload and SLO; then establish a baseline that meets the same correctness contract. Profile one saturated resource or critical path, choose one intervention whose mechanism matches it, and benchmark again under the same workload. Only then evaluate interactions with a second intervention. This sequence avoids attributing a gain to quantization when a changed batch policy created it, or claiming a speculative gain when the comparison quietly reduced output length. Modern serving systems such as PagedAttention-based memory management and Prefill–Decode disaggregation exemplify the value of treating capacity, request state, and phase objectives as measured system properties [4], [5].

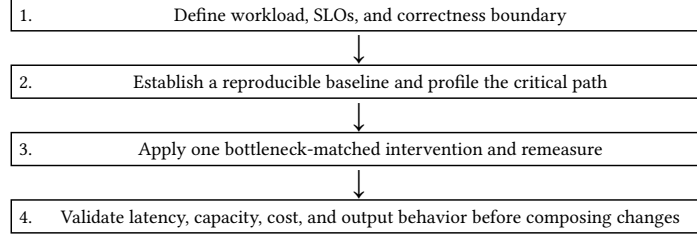


Figure 2: Inference optimization is an experimental loop. The workload and correctness boundary remain fixed while one proposed cause of the measured bottleneck is tested.

Common failures are violations of this loop. Optimizing a synthetic throughput benchmark instead of production-like request lengths can target the wrong phase. Reporting means but not p95 or p99 hides tail queueing and cache effects. Increasing batch size without observing TTFT can exchange one good metric for a failed product. Lower precision does not imply a faster kernel, and more GPUs do not imply lower latency when communication is exposed. Finally, multiple changes can interfere: a quantized target can change speculative verification cost; a cache policy can change the concurrency at which batching becomes useful; a distributed cache handoff can relocate rather than remove a bottleneck.

## 9 Implementation Contracts

The **workload contract** must version the model, tokenizer, prompt format, precision, hardware placement, sampling policy, prompt and output distributions, arrival process, concurrency, context and output limits, service classes, cancellation behavior, and SLO. It must state which events define request arrival, admission, first token, final token, and failure. A benchmark that changes any of these quantities is a new experiment, not a direct comparison.

The **measurement contract** must collect the same timestamps and resource counters under one clock. It should distinguish queueing from Prefill time, Prefill from Decode time, model-weight memory from KV Cache payload and allocator overhead, and compute from communication where the deployment is distributed. Percentiles must include the denominator population and a declared treatment of errors, cancellations, and timeouts. A dashboard that exposes only average tokens per second cannot establish an interactive-latency or cache-capacity claim.

The **correctness contract** must protect behavior while optimizing execution. It records the accepted numerical tolerance, deterministic prompt set or stochastic-evaluation protocol, stop semantics, output-quality threshold, and cache or distribution-preservation claims. It must be possible to associate any performance regression with a fixed checkpoint and runtime configuration, and any quality regression with the representation, kernel, or scheduling change that caused it. These contracts make the final serving configuration diagnosable, reproducible, and safe to evolve.

## 10 Summary

LLM inference is an end-to-end constrained optimization problem. Request distributions, queueing, Prefill, cache allocation, Decode, communication, completion, and reclamation jointly determine the latency, capacity, and cost experienced by a workload. TTFT, TPOT, end-to-end latency, throughput,

cache utilization, and p95/p99 tail behavior expose complementary properties; none independently establishes that a system is well designed.

Arithmetic intensity and Roofline-style reasoning help distinguish reductions in arithmetic from reductions in data movement, while memory-capacity, communication, and scheduling analysis supplies the constraints that such a model omits. KV Cache management, quantization, continuous batching, speculative decoding, and distributed inference are therefore conditional levers rather than a fixed stack. The durable methodology is to characterize the workload, define an SLO and correctness boundary, profile the active bottleneck, change one mechanism, remeasure under the same conditions, and compose only validated gains. This closes the Inference and Serving section with a system-design discipline that remains useful as models, hardware, and serving runtimes change.

## References

- [1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “ORCA: A Distributed Serving System for Transformer-Based Generative Models,” in *16th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/yu>
- [2] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009, doi: 10.1145/1498765.1498785.
- [3] R. Pope *et al.*, “Efficiently Scaling Transformer Inference,” *arXiv preprint arXiv:2211.05102*, 2022, doi: 10.48550/arXiv.2211.05102.
- [4] W. Kwon *et al.*, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, Association for Computing Machinery, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.
- [5] Y. Zhong *et al.*, “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2024, pp. 193–210. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>